TOMAS CORNEJO 105320237

MODIFICATIONS ON THE CODE

-Used _hypot() function (Euclidean distance)

-Modifications on min_edge_cost and enabled diagonal movements:

Edited README.txt with

Features CHANGES By Tomas

- Changed min_edge_cost from 10.0 to 1.0 in box_world.py

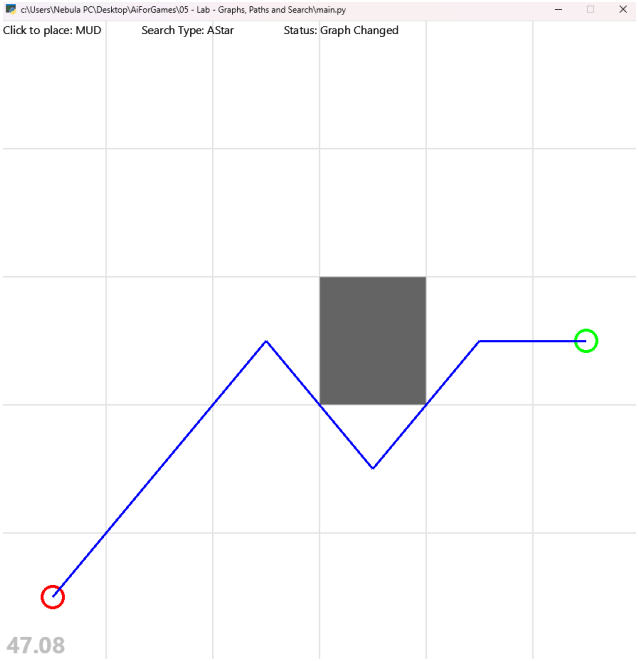
- Uncommented diagonal edge code in box_world.py

- Selected the active heuristic to _hypot() instead of _manhattan()

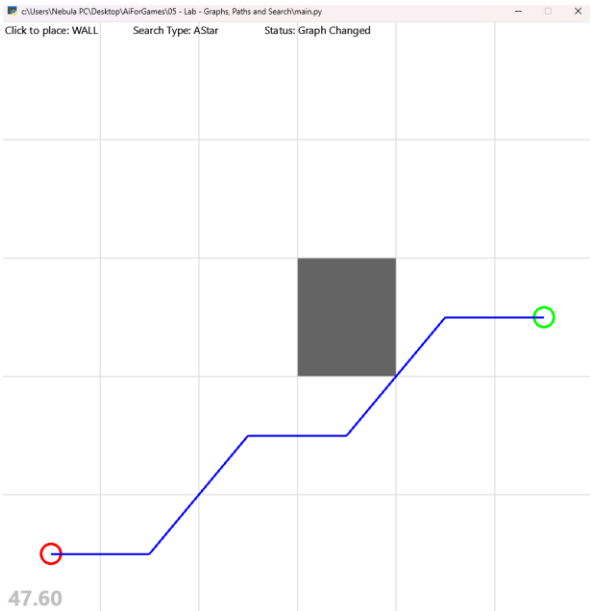
  - The Euchildean distance is better for diagonal movement estimates.

Represented with AStar

Main.py with map1 (suboptimal path with value 10)



Main.py with map1 (optimal path with value 1)



The modification on min_edge_cost is only going to affect heuristic-based searches, which in our cases will be AStar. Dijkstra also uses edge costs but is not affected by min_edge_cost since it uses the actual costs only.

Both BFS and DFS remain the same regardless of the change in min_edge_cost since they are not cost based, the only thing they look for is a connectivity path, there is no heuristic logic guiding the search.

First, I want to define, what is Path Cost?

> If you do the sum of the weight of all edges from your set Start to your set Finish you get the Path Cost, for example, moving only through clear cells will cost you 1.0, going from mud to another mud costs you 4.0, from water to mud costs you 9.0, from mud to clear costs you 2.0, etc.
>
> The algorithms AStar and Dijkstra care about these type of calculations since they try to minimize the total Path Cost, meanwhile, BFS and DFS do not care about this and just care about connect from Start to Finish.

What “Heuristic” means on AStar?

For AStar, the function h(n) takes the search towards the goal by estimating the REMAINING cost from a specific point to the Finish point. It takes the actual cost from the beginning to where it's "currently" standing and sums it with an "educated guess" towards the end from the "current" standing point.

By modifying the min_edge_cost function we are modifying the h(n) value which is the cost from the "current" point to the target"

C.- **In your lab notes, answer the following questions**: What does the diagonal edge code do? What is Manhattan distance? Where is it used in this codebase? What other heuristic functions exist for pathfinding, and how do they work?

-Thanks to the diagonal edge, we have a total of 8 available moves instead of just the regular 4.

Without diagonals if we are trying to find the path from Start to Finish we would have to do 4 movements

S-	-	|
		|
		F

1. East, 2. East, 3. South, 4. South

With the implementation of diagonal movement, we only need to do 2 movements

S \		
	\	
		F

1. South/East, 2. South/East

-Manhattan distance is a metric used to calculate the sum of absolute differences between coordinates. So if I have (0,0) and (2,4) the Manhattan calculation would look like |0-2| + |0-4| = 2 + 4 = 6. This is in the scenario of movement restricted to horizontal and vertical steps.

This formula is used on the box_world.py file in "def _manhattan(self, idx1, idx2):" to calculate the distance between nodes, then set as a Heuristic function on self.graph.cost_h = self._manhattan, and also used on the AStar algorithm that sums the cost from start to finish and the heuristic (which is the Manhattan distance multiplied by the min_edge_cost value.

- Other Heuristic functions are the Euclidean distance which calculates the straight line between 2 nodes by using the pythagorean theorem. This specific formula is useful for when the diagonals cost $\sqrt{2}$ for an any-angle movement situation. (This one is already implemented in the code with the _hypot() function).

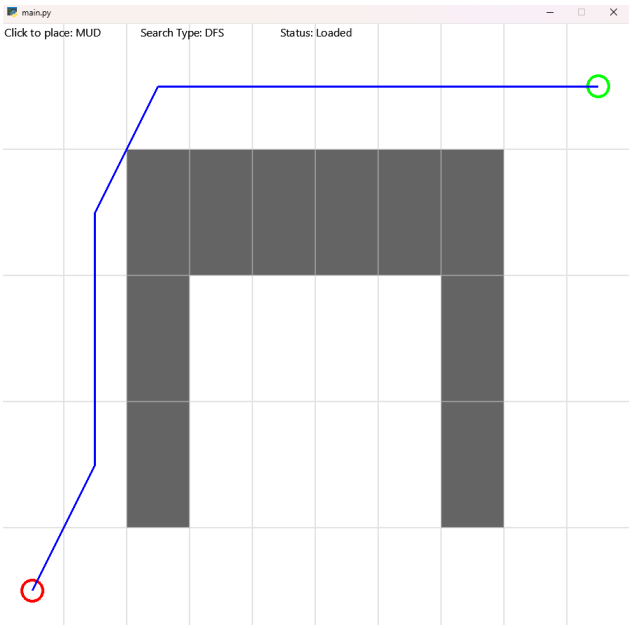
Another example could be the Chebyshev distance, it takes the MAXIMUM coordinate difference between 2 points and their absolute values. This formula is useful for when we have 8 possible directional movements, and the diagonals cost the same as the regular cardinal moves. (This one is already implemented in the code with the _max () function too).

Extensions:

Custom maps:

Map3.txt

DFS:

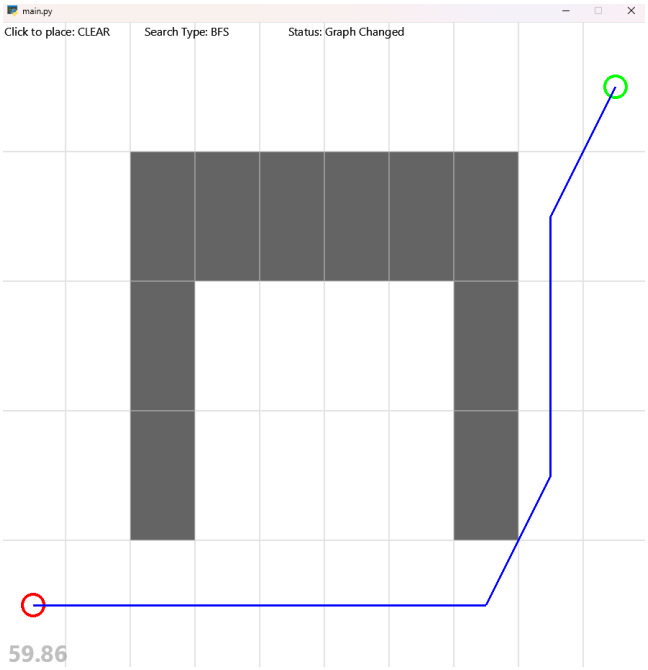


DFS is the only one that goes up in this scenario, all BFS, AStar, and Dijkstra go the bottom path.

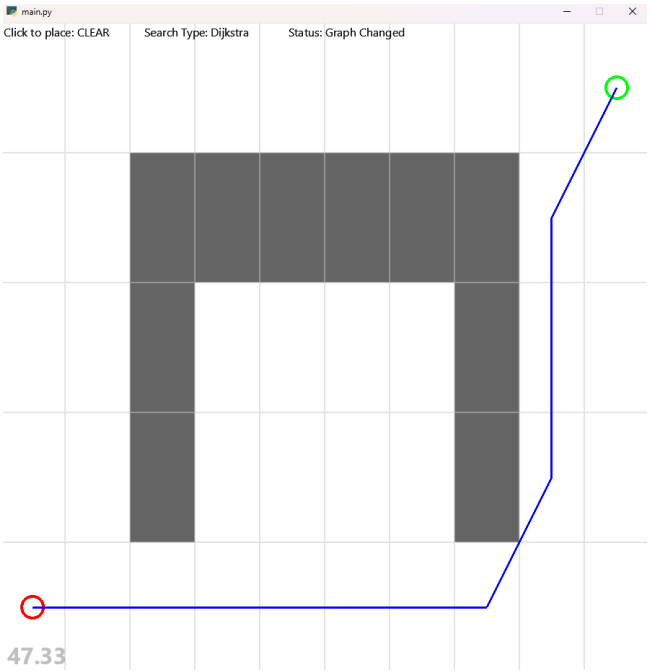
This helps to understand one of the main differences between DFS and BFS, the way that DFS search is written on the code, it will ALWAYS check top path first so it just stays with it, as the difference to BFS that explores outward all directions first.

Now, Dijkstra and AStar work on path finding that all have equal costs (they can get to the end without having to go through mud or water in this case) so they just return the first discovered path.

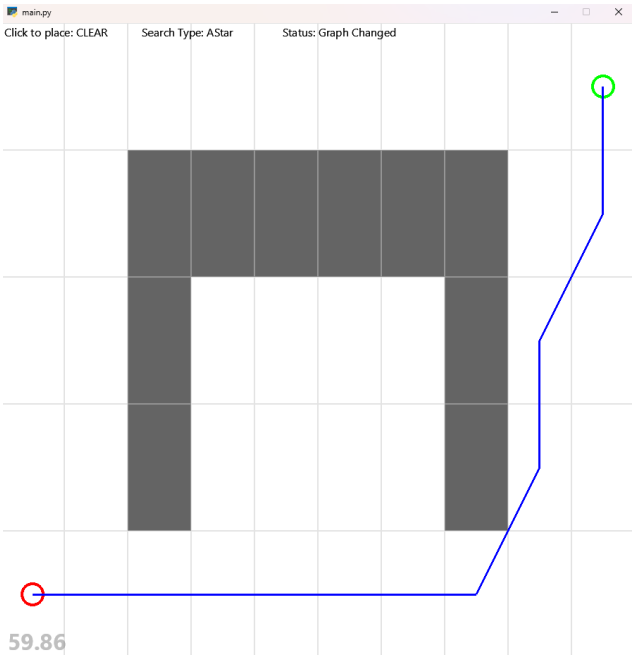
BFS:



Dijkstra:

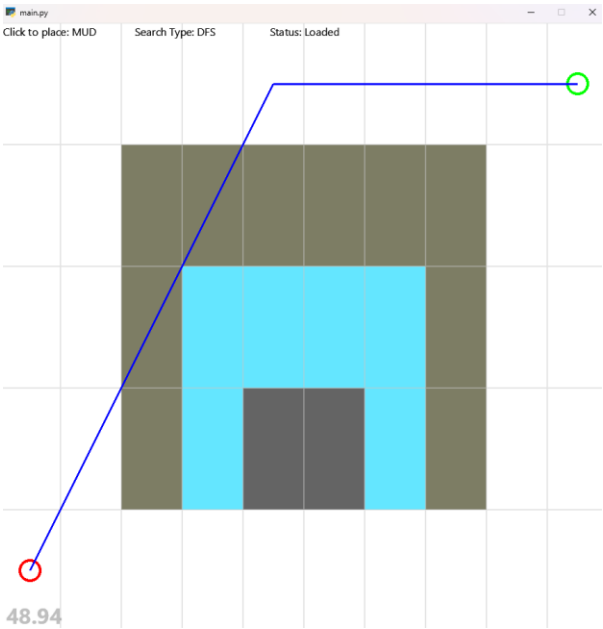


AStar:



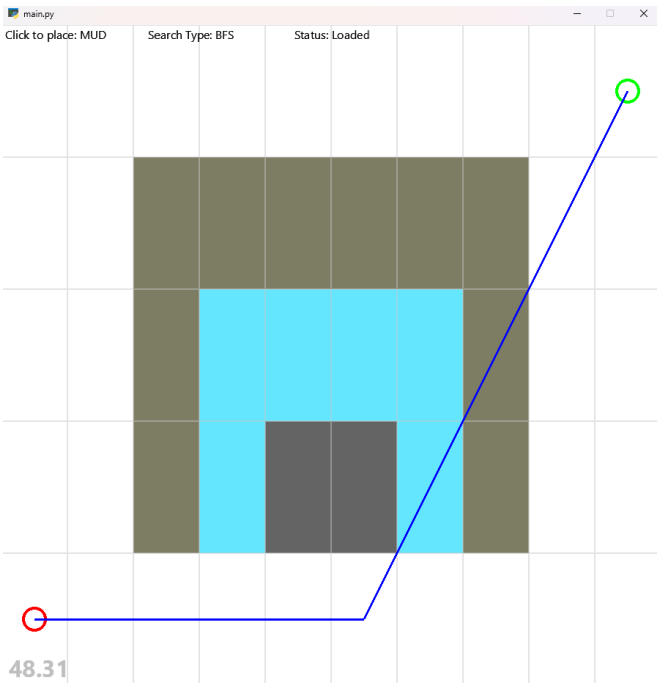
Map4.txt

DFS:



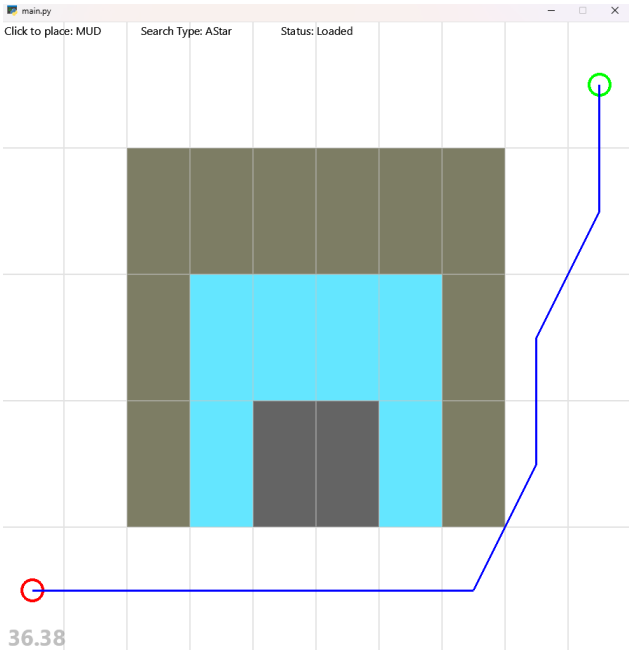
DFS doesn’t care about cost, so it will just find a path regarding on edge-checking. It will fully commit to the first available direction (UP in this case because of the checking order) towards the goal and stay with it.

BFS:

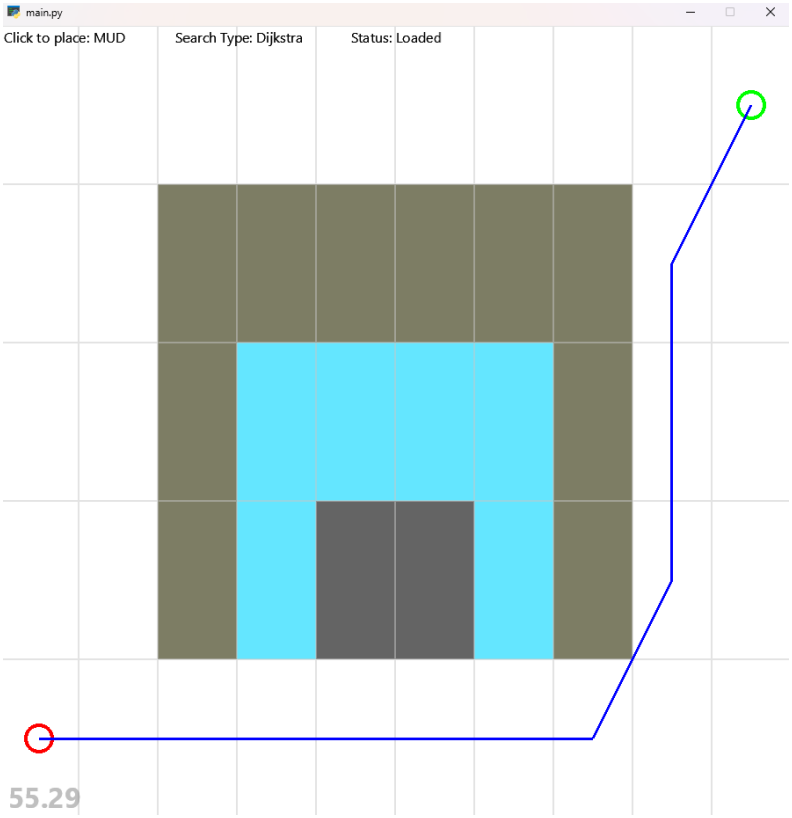


BFS discovers the diagonal shortcut by thinking on the best STEPS to take.

AStar:


main.py
Click to place: MUD
Search Type: AStar
Status: Loaded
36.38

Dijkstra:


main.py
Click to place: MUD
Search Type: Dijkstra
Status: Loaded
55.29

Both AStar and Dijkstra can find the most cost-efficient path towards the goal, the only difference is that AStar does it by the heuristic value on each movement regardless of the order, Dijkstra checks UP movements before DIAGONAL, which leads to that slight difference at the end.